



Getting Started with Ninjadini Debug Console

⚠️ Unity 2022.3 or newer is required

NjConsole relies on Unity's UI Toolkit, which became stable for runtime use in 2022.3 LTS.

✅ Nothing to set up

NjConsole starts itself — no prefab, no GameObject, no init call. Import it and it's running.

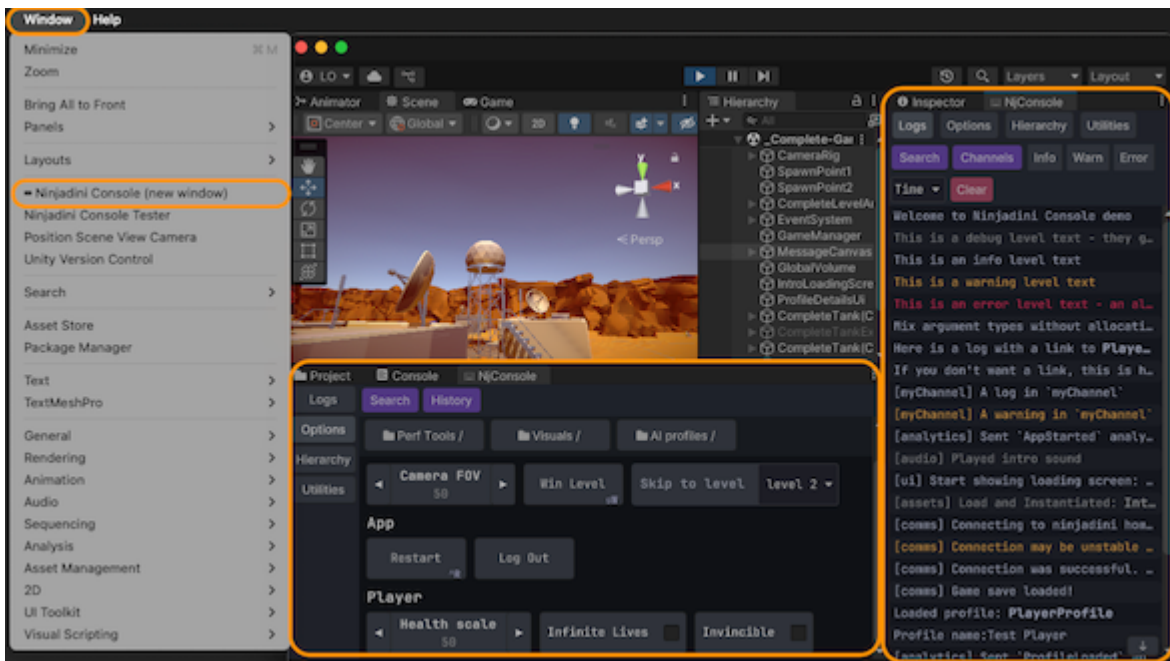
- 🎬 **Try the demo:** open `Demo/Demo.unity`. Delete the `Demo` folder once you're done with it.

Using the new Input System? Select the scene's **EventSystem** and click **Replace with InputSystemUIInputModule** in the Inspector.

- ⚙️ **Settings:** Project Settings > Ninjadini = Console
- 🌐 **Full docs:** ninjadini.github.io/njconsole

🧩 Open console window in editor

- Navigate to: `Window > = Ninjadini Console (new window)`
- Select it again to open another window — you can have several open at once.



📱 Open console in game view

- Keyboard:** Press the ``` key (top-left on US keyboard). Press again to close.
- Keyboard:** `Shift` + ``` opens straight into the Command Line.
- Mouse:** Hold at the top-left corner of the game screen for 1 second.
- Mouse (alternative):** Double-tap the top-left corner of the screen.

Customize these triggers in `Project Settings > Ninjadini = Console` — also where you set a **passphrase challenge** to prevent unintended access.

Sidebar Tips (During Play Mode)

- Press and hold any empty area of the sidebar to temporarily peek behind the console.
- On screens 650px or wider (or taller), a window mode button at the end of the sidebar lets you undock and resize the overlay.

The panels

Panel	What's in it
Logs	Log viewer, filtering, search, and the Command Line
Options	Your cheats / debug menu
Hierarchy	Live scene tree — drill into GameObjects and their components
Utilities	FPS & memory monitors, PlayerPrefs editor, editable screen & quality settings, device info, log export, UI scale

Clicking an object anywhere — a log link, a hierarchy entry — opens the **Object Inspector**: read and edit fields and properties, follow references, call methods, all while the game runs.



NjLogger

`Debug.Log` allocates and captures an expensive stack trace on every call. NjLogger:

- Formats arguments with zero allocations
- Integrates with NjConsole's filtering, channels and object inspection
- Still captures `Debug.Log()` automatically

Do I have to replace my `Debug.Log()` calls? No — they show up in NjConsole either way. Reach for NjLogger where the allocations matter. The two mix freely.

```
// These logs will appear in NjConsole with appropriate severity styling
NjLogger.Debug("This is a debug level text – they get auto excluded in release builds");
NjLogger.Info("This is an info level text");
NjLogger.Warn("This is a warning level text");
NjLogger.Error("This is an error level text – an alert shows when an error is logged.");

// Mixing types? No problem. This won't allocate.
NjLogger.Info("Mix argument types without allocation... integer:",123,"float:", 123.45f," bool:", true);

// Link to objects for runtime inspection
var playerObj = GetTestPlayerObj();
NjLogger.Info("Here is a log with a link to ", playerObj.AsLogRef(), " – you can inspect it");

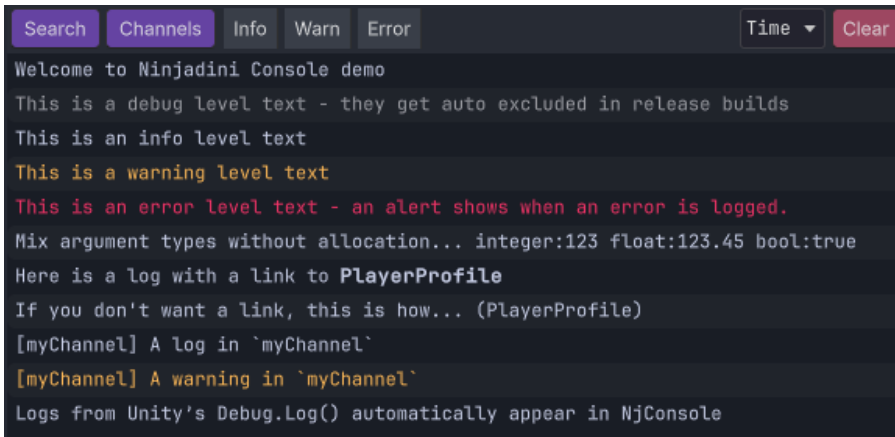
// Log object as plain string (no link)
NjLogger.Info("If you don't want a link, this is how... ", playerObj.AsString());

// Use named channels to group logs (recommended to keep as static readonly)
static readonly LogChannel channel = new LogChannel("myChannel");

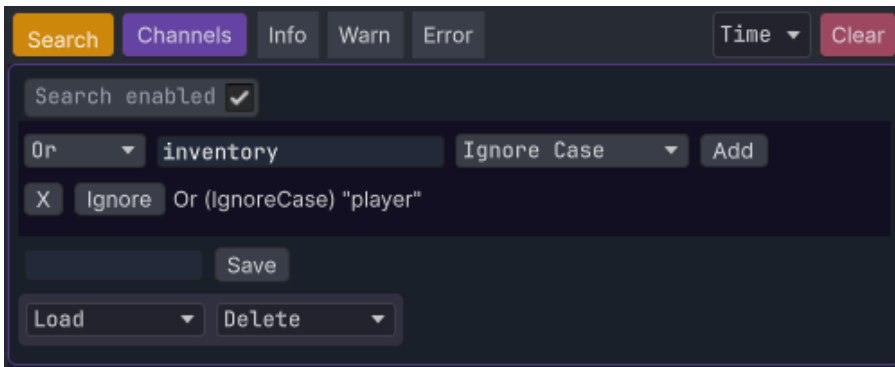
channel.Info("A log in `myChannel`");
channel.Warn("A warning in `myChannel`");

NjLogger.Info(Color.cyan, "Passing a Unity Color as the first param will auto color the log to that color.");

// Unity's native logs still show up in NjConsole
Debug.Log("Logs from Unity's Debug.Log() automatically appear in NjConsole");
```



Log filtering



-  **Search** — stack multiple conditions.
 - **And** all must match · **Or** at least one must match · **Not** must not match
 - Match types: **IgnoreCase** , **CaseSensitive** , **Loose** (fuzzy), **RegExp** . Term sets can be saved by name.
-  **Channels** — [*] all logs · [-] only logs with no channel
-  **Levels** — Debug, Info, Warn, Error, with warning / error counts on the button

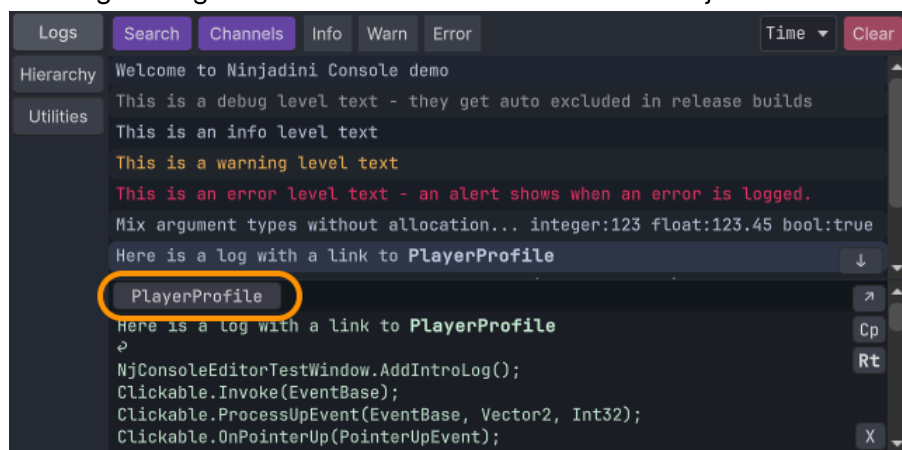
 Right-click any log to pin it. Pinned logs remain visible regardless of filters.

Logs object linking

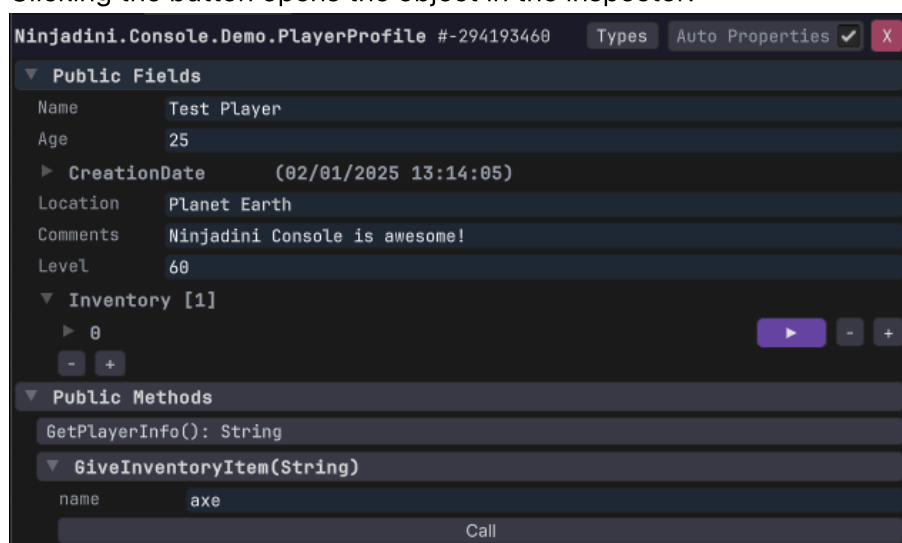
Log object references directly — click one to open it in the Object Inspector and edit it live.

```
var playerObj = GetTestPlayerObj();
NjLogger.Info("Here is a log with a link to ", playerObj);
NjLogger.Info("Here is a link to ", playerObj.AsLogRef(), " - mixed in multiple arguments");
```

Clicking the log row shows a button for each linked object:



Clicking the button opens the object in the inspector:

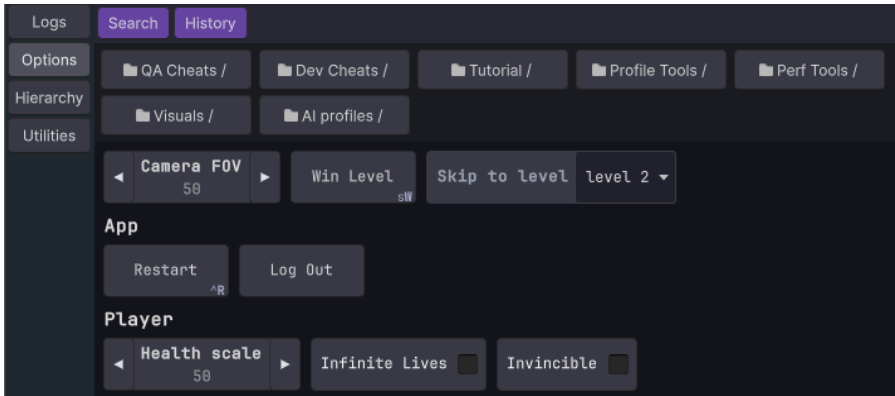


 **Memory-safe** — links are weak references, so they never leak. The trade-off: a collected object's link expires. To hold it for as long as the log stays in the ring buffer, use `aTestObj.AsStrongLogRef()`.

 Many fields and properties are editable, but not all types are supported yet.

Options Menu / Cheats

Build interactive menus for in-game tools, debug settings and cheats — great for prototyping and QA.



Two Ways to Add Options

Add Option Items with `[ConsoleOption]` Attribute

Register fields, properties or methods as menu items. Call this once during setup:

```
void Start()
{
    NjConsole.Options.CreateCatalogFrom(this, "TestOptions");
    // ^ second param `TestOptions` is optional, it puts all the items
    inside the `TestOptions` folder in this example.
    // If 'this' is a MonoBehaviour, options will auto-remove when
    `OnDestroy()`
}
```

 For static members pass the type instead:

`NjConsole.Options.CreateCatalogFrom(typeof(DemoNjConsole));` They're separate because statics persist without a live instance.

Buttons

```
[ConsoleOption]
void SayHello() {
    Debug.Log("Hello");
}

// directory / folder
[ConsoleOption("ChildFolder/My Second Button")]
void AnotherButton() {
    Debug.Log("Clicked my second [ConsoleOption] button");
}

// header
[ConsoleOption("A button inside a header",
    header:"My Header")]
void AButtonInsideHeader() {
    Debug.Log("Clicked my second [ConsoleOption] button");
}

// key binding - Shift + W to call WinLevelCheat() in playmode
[ConsoleOption(key:Key.W,
    keyModifier:ConsoleKeyBindings.Modifier.Shift)]
void WinLevelCheat() {
    Debug.Log("Clicked WinLevelCheat");
}

// auto close console overlay
[ConsoleOption(autoClose:true)]
void AutoCloseConsole() {
    Debug.Log("Console overlay should be closed now that you clicked a
button with auto close flag");
}
```

key: takes `UnityEngine.InputSystem.Key` on the new **Input System**, `UnityEngine.KeyCode` on the legacy **Input Manager**. Key bindings are Editor-only until you enable **Features > In Player Key Bindings**.

✔ Toggles

```
[ConsoleOption()]
bool InfiniteLives;

[ConsoleOption()]
bool InfiniteAmmo {get; set;}
```

Warning: Keybinding feature only works for buttons and toggles.

Numbers

```
[ConsoleOption]
int Health;

[ConsoleOption]
int HealthProperty {get; set;}

// With left and right step buttons
[ConsoleOption(increments:0.5f)]
float Speed;

// Range clamping
[ConsoleOption()]
[Range(1, 5)] // FYI: If you use a version before Unity 6, RangeAttribute can
not be used in properties
int Strength;
```

Text fields

```
[ConsoleOption]
[Multiline] // if you need multiline text entry, put [Multiline] attribute.
string UserCommentMessage;

[ConsoleOption]
void SaySomething(string receivedText)
{
    Debug.Log("You said: " + receivedText);
}
```

Enum Dropdown

```
[ConsoleOption]
DeviceOrientation preferredOrientation;
```

Command Line fallback for unsupported options

Members the Options Menu can't render — multiple parameters, unsupported types — are hidden, but still callable from the Command Line. You'll see a "3 hidden item(s)" notice with a button to open it.

Add Option Items Programmatically

For full control and dynamic setup.

```
var catalog = NjConsole.Options.CreateCatalog();
```

A catalog is the unit of cleanup — `catalog.RemoveAll()` when you're done with the set.

Buttons

```
catalog.AddButton("My First Button", () => Debug.Log("Clicked my first button"));

// directory / folder
catalog.AddButton("A Folder / Child Folder / Child Button", () => Debug.Log("Child button was clicked"));

// header sub-grouping
catalog.AddButton("A button in a header sub-group", () => {})
    .SetHeader("My Header");

// key binding to space key
catalog.AddButton("My Space Key Bound Button", () => Debug.Log("Clicked my Space key bound button"))
    .BindToKeyboard(KeyCode.Space);

// auto close console overlay
catalog.AddButton("My auto close button", () => Debug.Log("Console overlay should be closed now that you clicked a button with auto close flag"))
    .AutoCloseOverlay();
```

✓ Toggles

```
var toggle1 = false;
var toggle2 = false;

catalog.AddToggle("My First Toggle", (v) => toggle1 = v, () => toggle1);

// folder + key binding + auto close
catalog.AddToggle("A Folder / My T key Bound Toggle", (v) => toggle2 = v, () => toggle2)
    .BindToKeyboard(KeyCode.T)
    .AutoCloseOverlay();
```

Buttons and toggles take `.BindToKeyboard(KeyCode.Space)`, or a combo via `.BindToKeyboard(KeyCode.E, ConsoleKeyBindings.Modifier.Shift | ConsoleKeyBindings.Modifier.Ctrl)`. One keybinding per item. `.AutoCloseOverlay()` closes the overlay after a press.

1234 Numbers, text & dropdowns

```
var aFloat = 12.34f;
catalog.AddNumberPrompt("A Number", (v) => aFloat = v, () => aFloat);

// clamped, plus left/right step buttons via the last arg
var stepped = 50;
catalog.AddNumberPrompt("0 to 100", (v) => stepped = Mathf.Clamp(v, 0, 100), ()
=> stepped, 2);

var text = "Initial text";
catalog.AddTextPrompt("My Text Prompt", (v) => text = v, () => text);

// Setter before getter here too.
var choices = new List<string>() { "A", "B", "C", "D" };
var index = 0;
catalog.AddChoice("A Choice List", choices, (v) => index = v, () => index);

var platform = RuntimePlatform.OSXEditor;
catalog.AddEnumChoice("A Choice Enum", (v) => platform = v, () => platform);
```

`AddTextPromptWithValidation()` lets you reject or rewrite input before it's accepted — see the online docs.

💡 Grouped paths keep menus navigable: `catalog.AddButton("App / Utilities / Reload Scene", () => ReloadScene());`

🤔 Should You Use `[ConsoleOption]` or Add Programmatically?

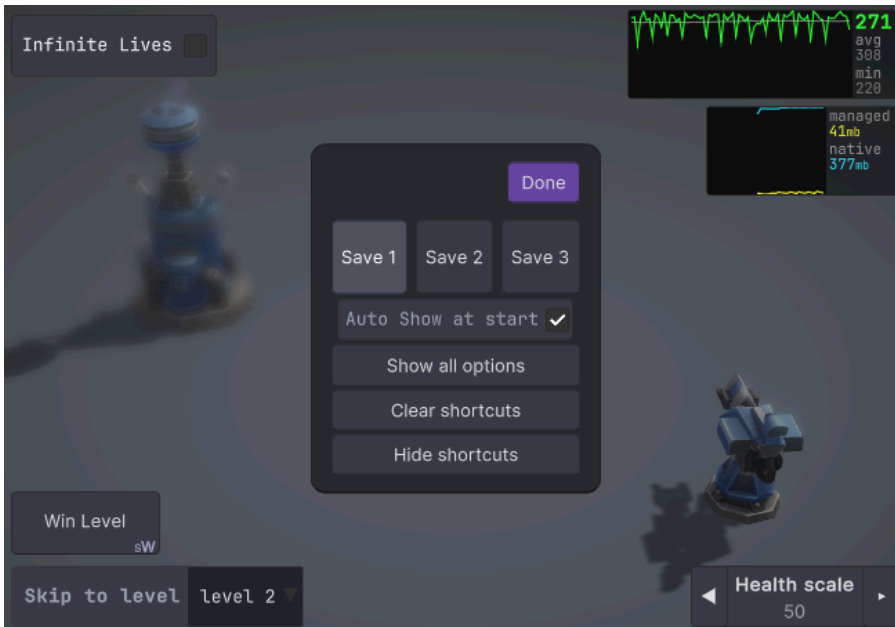
[ConsoleOption] — quick and declarative, and cleans up automatically when the `MonoBehaviour` is destroyed.

Programmatic — control over when and how options appear, slightly faster (no reflection), and the only way to build options dynamically:

```
foreach (var itemType in inventoryItemTypes)
{
    var local = itemType;
    catalog.AddButton("Inventory/Give " + itemType.Name, () =>
GiveItem(local));
}
```

Shortcuts

Pin any option item or folder for quick access. Two kinds: **menu shortcuts** (a pinned list at the top of the Options panel, editor window and runtime overlay) and **overlay shortcuts** (buttons snapped to a screen corner, runtime overlay only). Below covers overlay shortcuts.



How to Create a Shortcut

Press and hold any option item or folder, then drag it to a screen corner.

Shortcuts snap to one of four corners and fill horizontally or vertically depending on where you drop — top-left can fill right or down.

Shortcut Edit Mode

Dropping your first shortcut enters edit mode:

-  Drag existing items to reposition them.
-  Switch between 4 layouts (Set 1 – Set 4).
-  Auto Show at start displays them on game launch.
-  Shortcuts follow their linked options — they appear and disappear together.
-  After Hide shortcuts , reopen edit mode with the ☐ Shortcuts button in the Options panel.



Command Line

Run your own and built-in commands as text. Commands register exactly like the **Options Menu** — via `[ConsoleOption]` or programmatically.



Basic Usage

- **Show:** press any key while focused on the Logs panel, or `Shift + `` in the runtime overlay.
- **Autocomplete:** `Tab` accepts the first suggestion; `Shift+↑` / `Shift+↓` navigates, `Tab` or `Enter` accepts.
- **History:** `↑` / `↓`.
- **Hide:** `Esc`.



On Mobile (no physical keyboard)

Tap the **Logs** button again to show it. The field uses a text-prompt style with autocomplete — tap  to switch between prompt and normal input.



Command Structure & Syntax

```
<command> <parameters separated by space ` ` or comma `,`>
```

Built-in commands live under `/` — `/help` lists everything. **String** params take quotes (`"`) for spaces or commas; escape with `\`.

- Names are case-insensitive. Casing conflicts show in autocomplete but may not execute correctly.
- Names can contain spaces; `/` creates folders, as in the Options Menu.
- Overloads are **not** supported — each command must be unique.

💡 Examples

```
[ConsoleOption]
void SayHello() {
    Debug.Log("Hello");
}
```

Command: sayhello

```
[ConsoleOption("profile/name")]
public string Name;
```

Get Command: profile/name

Set Command: profile/name "My name here"

```
[ConsoleOption("demo/introduce")]
void IntroducePerson(string name, int age) { ... }
```

Command: demo/introduce Ninjadini 30

or demo/introduce "Ninjadini", 30

🚧 Advanced Parameters - Constructor Arguments

Group constructor arguments in parentheses:

```
[ConsoleOption("math / vector multiply")]
static Vector3 MultiplyV(Vector3 a, float b) => a * b;
```

Command: math/vector multiply (1 2 3) 1

or math/vector multiply (1,2,3),1

More in the [online documentation](#): nested constructors, return values, storage commands and `$` variables, scopes, accessing logged and hierarchy objects, separating command catalogs, input prompt takeover, and custom executors.



Before you ship

Keep players out

Add an access challenge so the console can't be opened by accident: `Project Settings > Ninjadini`
`= Console > Playmode Overlay > Add Access Challenge > Secret Pass`. Individual panels can also be switched off for player builds under `... > Features`.

Or strip it out entirely

`Project Settings > Ninjadini = Console > Disable NjConsole` adds the `NJCONSOLE_DISABLE` define and compiles NjConsole away — smaller build, no memory cost, nothing to reverse engineer. Key classes stay stubbed so your project still compiles. Recommended for production releases.

Wrap your own cheat code in `#if !NJCONSOLE_DISABLE` so it's stripped too.



Cheat sheet

Key	Where	Action
<code>`</code>	Game view	Show / hide the console overlay
<code>Shift + `</code>	Game view	Open straight into the Command Line
any key	Logs panel	Open the Command Line
<code>Tab</code>	Command Line	Accept the first suggestion
<code>Shift + ↑ / ↓</code>	Command Line	Move through suggestions
<code>↑ / ↓</code>	Command Line	Previous commands
<code>Esc</code>	Command Line	Close it
<code>Ctrl / Cmd + C</code>	Logs panel	Copy selected rows
<code>Ctrl / Cmd + Shift + C</code>	Logs panel	Copy rows <i>with</i> stack traces
Right-click	A log row	Pin it — stays visible through filters



If something looks off

Overlay is the wrong size (on device, or in Game view on a high-DPI screen) — `Utilities > Tools > UI Scale + / -`.

Errors after importing — on the new Input System? Install the Input System package, or set `Project Settings > Player > Active Input Handling` to `Input Manager (old)`.

Key bindings don't work in a build — they're Editor-only until you enable `... > Ninjadini = Console > Features > In Player Key Bindings`.

Logs appear twice — your own logger is likely feeding both `Debug.Log()` and NjLogger. Turn off `... > Ninjadini = Console > Logging > Debug.Log()` to NjLogger.



Ready for More?

📖 For advanced topics such as:

- Creating custom modules and panels
- Advanced Command Line features
- Building editor-bound options menus
- Accessing log history and writing custom log handlers
- Customizing log timestamp formats
- Creating your own access challenge or console activation trigger

...and more!

👉 Refer to the online manual for full documentation:

<https://ninja dini.github.io/njconsole/>